Ders 12 Çalışma Özeti

Ders 12 Çalışma Özeti

Genel Konular

- Memory Networks'ün tekrarı (ekrandan, 00:10:00 ve 00:11:00)
  - Elimizde bir bellek (memory, hikâyenin cümleleri) ve bir soru vardır. Mekanizma, sorunun cevabının bellekteki hangi bloklardan yararlanılarak bulunacağını seçer.
  - Slayttaki tanımlar: m hikâyenin tutulduğu bellek; m_{o_1} ve m_{o_2} belleğin cevabı üretecek parçaları; x soru; r tek kelimelik cevap. Bellek yalnız yazılır ve okunur, değiştirilmez.
  - Çıkarım iki adımlıdır: $o_1 = \arg\max_i s_O(x, m_i)$, $o_2 = \arg\max_i s_O([x, m_{o_1}], m_i)$, cevap $r = \arg\max_{w \in W} s_R([x, m_{o_1}, m_{o_2}], w)$.
  - Benzerlik: $s(x, y) = \Phi_x(x)^\top U^\top U \Phi_y(y)$. Φ (teta) fonksiyonu metni $d \times 1$ boyutlu bir BoW gösterimine çevirir; U ($n \times d$) eğitimde öğrenilir. s_O ve s_R'nin U matrisleri farklıdır.
  - Son adımda W bellek bloklarından değil sözlükten (vocabulary) oluşur. bAbI görevlerinde sözlük çok küçüktür (yüz civarında kelime).
  - Kayıp fonksiyonu: soru ile doğru destekleyici cümle (örnekte 6, sonra 2) ve doğru cevap ("bahçe") arasındaki benzerliğin yüksek, diğer bütün cümle ve kelimelerle düşük olması istenir. Eğitim kümesindeki "6 ve 2" bilgisi bu kayıpta kullanılır.
- End-to-End Memory Networks (ekrandan, 00:16:00 ve 00:20:00)
  - Amaç, eğitim kümesindeki destekleyici cümle indislerine (6 ve 2) olan ihtiyacı kaldırmaktır.
  - Belleğin iki temsili vardır (A ve C gömmeleri). Soru B ile gömülür (u). "En benzeyeni seç" (tek-sıcak, one-hot seçim) yerine benzerlik oranında ağırlıklar p_i hesaplanır.
  - Çıkış $o = \sum_i p_i c_i$, cevap $\hat{a} = \text{Softmax}(W(o + u))$. Eğitimde A, B, C, W gömme matrisleri öğrenilir.
  - Tek adımlı ve üç adımlı (her katmanda A^k, C^k) mimariler vardır. Her katmanda bellekteki cümlelerin soruya göre etkinliği değişir.
  - Konum gömmeleri (position embeddings): BoW gösterimi kelime sırasını önemsemez. Çözüm $m_i = \sum_j l_j \cdot A x_{ij}$ (element bazlı çarpma), $l_{kj} = (1 - j/J) - (k/d)(1 - 2j/J)$. $d = 128$, $J = 25$ için L'nin her sütunu bir indeksi gösterir; ardışık sütunlar birbirine yakın, uzak sütunlar farklıdır.
  - Hoca bu yapının Transformer'daki dikkat (attention) mekanizmasını andırdığını söyledi: iki temsil (anahtar ve değer gibi), benzerlik ağırlıkları ve ağırlıklı toplam. Self-attention'da bir kelimenin anlamı, çevresindeki kelimelerin vektörlerinin ağırlıklı ortalaması gibi çıkar.
- Bulunan sonuçlara itiraz: kısa yol öğrenme (ekrandan, 00:20:45 ve 00:22:40)
  - 20 görevde (bAbI) her adımda (hop) hangi cümleye ne kadar dikkat edildiği tablolarda gösterilir; örnek: "Where is John? Answer: bathroom, Prediction: bathroom".
  - İtiraz (arXiv:1808.04926v1): 5 veri kümesi üzerinde 3 modelin başarısı; orijinal veriyle, yalnız metinle ve yalnız soruyla ölçülmüş. Bazı veri kümelerinde eksik hallerle de orijinale yakın sonuç alınmış.
  - Sonuç: adı X olan bir veri kümesinde başarılı olan bir sistem X işini yapabiliyor demek değildir. Sistem sorudan ya da metinden cevaba, bizim istemediğimiz ve düşünmediğimiz eşlemeler (mapping) yapmış olabilir. Hoca bunu önceki dersteki tank örneğine bağladı.
- QANet (ICLR 2018) (ekrandan, 00:24:45 ve 00:30:00)

  - Paragraf (C) n, soru (Q) m kelimedir. Her kelime GloVe ile 300, karakter tabanlı 200, toplam 500 boyutlu gömülür; gömme kodlayıcı bloklarından sonra $d = 128$ boyutlu olur.
  - Benzerlik $S_{ij} = W_0[c; q; c \odot q]$; W_0 $3d$ boyutlu öğrenilen bir vektör, S $n \times m$ matristir. $\bar{S}$ satır, $\bar{\bar{S}}$ sütun normalizasyonudur (softmax). $A = \bar{S} \cdot Q^T$, $B = \bar{S} \cdot \bar{\bar{S}}^T \cdot C^T$. Context-Query Attention çıkışı $4d \times n$ boyutlu $[C; A; C \odot A; C \odot B]$ olur.
  - Çıkış: cevabın paragrafta başladığı ve bittiği yerin olasılıkları. $p^1 = \text{softmax}(W_1[M_0; M_1])$, $p^2 = \text{softmax}(W_2[M_0; M_2])$, W_1 ve W_2 slayta göre $1 \times 8d$ boyutlu öğrenilen vektörler.
  - Kayıp: $L(\theta) = -\frac{1}{N} \sum_i \left[\log p^1_{y^1_i} + \log p^2_{y^2_i} \right]$. Test: $s \leq e$ ve $p^1_s p^2_e$ en büyük olan (s, e) ikilisi.
  - Yapay veri artırımı: çift çeviri (İngilizce → Fransızca → İngilizce, back translation).
  - Model üretici değildir; paragraftaki bir parçayı seçer. Bu SQuAD 1.0'ın yapısıdır; 2.0'da cevabı olmayan sorular da vardır.
- Seçmeden üretmeye geçiş (00:30:00)
  - BERT ince ayarında (fine-tuning) da görevler şıklar arasından seçme ya da paragrafta başlangıç-bitiş bulma biçimindeydi.
  - ChatGPT ile birlikte yöntem üretici süreçlere kaydı: cevap paragraftan kopyalanmaz, üretilir. Büyük modellerin hepsi otoregresif üretici süreçlerle çalışır.
- Otoregresif üretim (00:33:02)
  - Bir sonraki adımı (token) tahmin edip üretileni tekrar girdi olarak vererek uzun dizi üretilir: x_1, x_2 verilince x_3, sonra x_1, x_2, x_3 ile x_4.
  - Eski bir fikirdir; hoca geri yayılım (backpropagation) makalelerinde de (1980'lerin sonu) bir sonraki kelime tahmini problemlerinin olduğunu söyledi.
  - Hipotez: bir dizinin devamını tahmin edebiliyorsanız o diziyi üreten süreci büyük ölçüde kavramışsınızdır. Hoca "kavramak" kelimesinin insan anlamında alınmaması gerektiğini, makineye insan terimleri atfetmenin uygun olmadığını söyledi.
  - Bir sonraki token tahmini bir sınıflandırma problemidir (regresyon değil). Sınıflar karakterler (84) ya da tokenizer çıkışlarıdır (32 bin, 48 bin gibi). Farklı olan eğitim kümesidir: öz denetimli (self-supervised) yapıda insan emeği olmadan çok büyük veri üretilir.
- LSTM ile şiir üretimi (ekrandan, 00:36:45 ve 00:40:00)
  - Kod: `siir_uretimi.m`, `tum_siirler.txt`. Yaklaşık 4 milyon karakterlik şiirler; LSTM bir sonraki karakteri tahmin edecek biçimde eğitildi (many to many).
  - 100'den fazla geçen 84 karakter kullanıldı; her giriş 84 boyutlu one-hot vektör, her çıkış 84 sınıftan biri. Kelime gömmesi yok.
  - İlk 200 bin karakter, 512 LSTM hücresi, 500 iterasyon, yaklaşık 2 saat eğitim. Farklı başlangıçlarla örnek şiirler üretildi.
  - Slayttaki not: "Falcı 10 şey söyler 1'i tutar, falcı bildi olur."
- Üretici modellerin değerlendirilmesi (00:40:26)
  - En büyük problem değerlendirmedir: üretilmesi gerekeni tam söyleyemeyebilirsiniz; üretilen şey aynı anlamı başka biçimde ifade edebilir. Ölçeklenebilir değerlendirme sistemi gerekir. Hocaya göre bu, pekiştirmeli öğrenme (reinforcement learning) yapısıyla şık bir yolla çözüldü.
  - Üretilen şiirlerin hiçbiri bütün olarak iyi değildir ama içlerinde iyi parçalar vardır. Lokal iyilik kolay, bütünlük zordur.
  - İddia edilen hipotez: model ve (iyi) veri birlikte büyütüldükçe makinenin yeteneklerinde doygunluk sınırı olmadığı.
- Haber üretimi (ekrandan, 01:01:20)
  - Benzer teknik; 500 bin karakterlik spor haberleri; kod `haber_uretimi.m`, `haber42bin.mat`; 512 hücre, 30 iterasyon, 2 saat eğitim.
  - Üretilen örnekler lokal olarak anlamlıdır ("türkiye futbol federasyonu (tff) tarafından düzenlenen") ama bütün olarak bir haber değildir. Hoca bunu istatistiksel dil modellerindeki pencere problemine benzetti.
  - Transformer'lar veriyi adım adım değil paralel aldıkları için bağlam uzunluğu kadar geçmişi işe katabilir. Daha büyük model, Transformer ve aynı kayıp (next token prediction) ile bütünlük de büyük ölçüde sağlanır.
- Şiir ile haber üretiminin karşılaştırması (01:05:11)
  - Mimari ve veri büyüklüğü açısından fark yoktur. Şiir hata kaldırır: garip ifadeler sem-

bolik olarak yorumlanabilir (“Bahçem yaralı kapılar gibi”). Düz metinde hata hemen göze batar.
  - Şiirler çok daha çeşitlidir (iki şiir birbirine az benzer); spor haberleri belli bir kalıba sahiptir. Çeşitlilik, şiir üretiminin bir tık daha başarılı görünmesini sağlar.
- Diyalog sistemleri (ekrandan, 01:10:00)
  - İki tür: yalnız sohbet amaçlı (gerçeklemesi çok zor, çeşitlilik çok yüksek) ve hedefi olan (ürün/hizmet bulan, alan; kısıtlı uzay, daha kolay; ticari taraf).
  - ELIZA (1965, Joseph Weizenbaum): bir terapisti taklit eder. Uyan şablon varsa dönüşüm uygular, yoksa genel amaçlı bir cevap verir: `(X me Y) → (X you Y)`, `(I remember X) → (Why do you remember X just now?)`, `(My {family-member} is Y) → (Who else in your family is Y?)`, `(X {family-member} Y) → (Tell me more about your family)` (ekrandan, 01:12:10). Kullanıcının sözünü dönüştürüp geri söyler (Rogerian psikoterapi tekniği).
  - PARRY (Colby, 1971): paranoyak bir hastayı taklit eder; elle kodlanmış, emek isteyen şablon tabanlı bir yapıdır.
  - Elle kodlama pahalıdır ve bir sistemden ötekine aktarılamaz.
- Diyalog derlemleri ve benzerlik (ekrandan, 01:14:40 ve 01:16:15)
  - Derlem kaynakları: Twitter, WhatsApp, film/dizi diyalogları; kitap, ansiklopedi de olabilir. Kullanıcının söylediğine (q) derlemde (D) en benzeyen ifade $t' = \arg\max_{t \in D} \frac{q^T t}{\|q\|\|t\|}$ bulunur, cevap olarak t''ye verilen cevap döndürülür. SSS'lerden cevap bulmak da benzerdir.
  - One-hot ile kosinüs, “Cimbom kazandı” ile “Galatasaray yendi” arasında hiçbir benzerlik bulamaz. Soft cosine: $\cos_M(X, Y) = \frac{X^t M Y}{\sqrt{X^t M X}\sqrt{Y^t M Y}}$, $X^t M Y = \sum_i \sum_j x_i m_{ij} y_j$.
  - M için iki yöntem: $m_{ij} = \max(0, \cos(v_i, v_j))^2$ (word2vec vektörleri, anlamsal benzerlik) ve Levenshtein uzaklığına dayalı $m_{ij} = \alpha \left(1 - \frac{\text{Levenshtein}(w_i, w_j)}{\max(\|w_i\|, \|w_j\|)}\right)^\beta$ (yazım farkları, sözdizimsel varyasyon).
  - Bu sistemler hafızasızdır: önceki soru ile şimdiki soru bağımsızdır. SSS araması için uygundur, sohbet için değildir.
- RNN'lerle diyalog öğrenme (ekrandan, 01:20:00)
  - Diyalog bir tarihçedir; cevap, yalnız son söze değil önceki bütün konuşmaya bağlıdır. RNN/LSTM'in durumu (state) bu belleği sağlar (Memory Networks'teki bellekle karıştırılmamalı).
  - Kodlayıcı-çözücü (encoder-decoder): herhangi bir veri tipinden vektöre dönüşüm kodlayıcı, vektörden herhangi bir veri tipine (metin, ses, görüntü, video) dönüşüm çözücüdür. Slayttaki 3 cümlelik örnek: “Mom, i don't feel so good / What's wrong? / I feel I'm going to pass out” (arXiv:1507.04808).
- Microsoft Tay (ekrandan, 01:21:40)
  - Kullanıcılarla yaptığı diyalogları da derlemine ekleyen bir sistem; açıldıktan 16 saat sonra kapatıldı. Kötü niyetli kullanıcıların söylemleri botun söylemlerine temel oluşturdu.
  - Hoca, ChatGPT'de kullanıcı girdilerinin anında eğitime girmediğini, filtrelerden geçtiğini söyledi.
- Sanal asistanlar (ekrandan, 01:24:10 ve 01:29:20)
  - Hedef odaklı, görev tabanlı, çerçeve tabanlı (goal oriented, task based, frame based). Genel yapı 1977'den beri değişmedi: GUS.
  - İşlem türleri belirlenir, her tür için gerekli bilgiler (slot) ve türleri (zaman, sayı, şehir) belirlenir, bu bilgileri almak için sorular belirlenir, bilgiler bir sırayla alınır.
  - Örnek: “Ankara'ya giden uçuşlar hangileri?” → Niyet: uçuş sorgulama; kalkış şehri: kullanıcının şehri → İstanbul; kalkış tarihi ve zamanı: ??; varış şehri: Ankara. Boşluklar için soru: “İstanbul'dan Ankara'ya hangi tarih için uçuş arıyorsunuz?”
  - Ticari sistemler güvenlik (safety) yüzünden üretici modellere yavaş geçer: %99 başarı bile kabul edilmeyebilir; “yapamıyorsa yapmasın ama yanlış yapmasın” yaklaşımıyla katı, geleneksel yapılar kullanılır.

Hocanın Özellikle Vurguladığı Kısımlar

- Bir veri kümesindeki başarı o işin çözüldüğü anlamına gelmez
  - Adı X olan veri kümesinde başarılı olan makine X işini yapıyor demek doğru değildir. Yönlendirmeye çalışsak da model istemediğimiz kısa yollar bulabilir.
- Yenilikler bir anda ortaya çıkmaz
  - Memory Networks'teki iki temsil, ağırlıklı toplam ve konum gömmeleri Transformer ve BERT'e giden adımlar olarak okunabilir. Attention NLP'den çıkmış bir kavram değildir; Transformer'ın farkı self-attention'ı öz denetimli görevlerde çok büyük ölçekte uygulamasıdır.
  - Geçmişe bakınca her şey anlamlı görünür; ama o anda doğru yönü seçmek şans ve öngörü ister.
- Bir sonraki token tahmini bir sınıflandırma problemidir
  - Yapı aynı kalır; farkı yaratan öz denetimli olarak çok basit kodla üretilen dev eğitim kümesidir.
- Üretici modellerin asıl problemi değerlendirmedir
  - Ölçeklenebilir değerlendirme gerekir; lokal iyilik kolay, bütünlük zordur.
- Ticari diyalog sistemlerinde hata kaldırılmaz
  - Bireysel kullanımda modelin hatasına gülersiniz; bankanın chatbotu para kaybettirdiğinde gülmezsiniz. Bu yüzden ticari taraf katı ve yavaş ilerler.

Kısa Tekrar Notları

- Memory Networks: o_1, o_2 ve r argmax ile seçilir; $s(x, y) = \Phi_x(x)^\top U^\top U \Phi_y(y)$; destekleyici cümle indisleri kayıpta kullanılır.
- End-to-End MemN2N: yumuşak ağırlıklar p_i, $o = \sum_i p_i c_i$, $\hat{a} = \text{Softmax}(W(o + u))$; destekleyici indis gerekmez; çok adımlı; konum gömmeleri.
- İtiraz (arXiv:1808.04926): yalnız metin ya da yalnız soru ile bazı veri kümelerinde orijinale yakın başarı.
- QANet: context-query attention, başlangıç ve bitiş olasılıkları, back translation ile veri artırımı.
- Seçici modellerden üretici (otoregresif) modellere geçiş.
- Next token prediction = sınıflandırma; öz denetimli büyük veri.
- LSTM şiir üretimi: 84 karakter, one-hot, 512 hücre, 2 saat; haber üretimi: 500 bin karakter spor haberi.
- Şiir hata kaldırır ve çeşitlidir; haber kalıplıdır ve hata göze batar.
- Diyalog: sohbet amaçlı (zor) ve hedefli (kolay, ticari). ELIZA (1965), PARRY (1971).
- Derlem + kosinüs, soft cosine (M: word2vec ya da Levenshtein).
- RNN encoder-decoder ile tarihçeli diyalog; Microsoft Tay 16 saatte kapatıldı.
- Sanal asistanlar: GUS (1977), niyet + slot doldurma.

Detaylı Açıklamalar

- Memory Networks'te eğitim kümesi yalnız cevabı değil, cevabın hangi bellek cümlelerinden bulunacağını da (bAbI örneğinde 6 ve 2) verir. Tasarımcılar "cevabı nasıl bulursan bul" demek yerine modeli önce ilgili parçaları bulmaya, sonra onlarla ilerlemeye yönlendirmek istemiştir. Kayıp fonksiyonu bu üç seçimin (birinci cümle, ikinci cümle, cevap kelimesi) her birinde doğru olanın benzerliğini yüksek, geri kalanların (formülde üstü çizgili olanlar) benzerliğini düşük tutar.
- End-to-End Memory Networks'te seçim yumuşaktır: soru gömmesi u ile A temsilindeki her bellek cümlesinin benzerliği softmax'tan geçirilip p_i ağırlıkları bulunur, sonra C temsilinin ağırlıklı toplamı o alınır. Hoca bunu Transformer'daki anahtar-değer yapısına benzetti: bir kelimenin anlamı çevresindeki kelimelerin ağırlıklandırılmış ortalamasından gelir. Katman sayısı arttıkça (1, 2, 3 adım) her katmanın kendi A^k, C^k temsilleri olur.
- Konum gömmelerinde kelimenin kendisi 128 boyutludur ve 128 boyutlu sabit bir konum vektörü ile element bazında çarpılır. Formülle hesaplanan L matrisinde ardışık sütunlar birbirine yakın, uzak sütunlar farklıdır; böylece aynı kelimenin cümleye etkisi farklı indislerde farklı olur.

- QANet'te soru cevaplama, paragraftaki her token için başlangıç ve bitiş olasılığı hesaplamaya dönüşür. Model sözlükten kelime seçmez, paragraftaki bir aralığı seçer; bu yüzden üretici değildir. Günümüzde ise cevap paragraftan kopyalanmadan üretilmektedir.
- Otoregresif üretimde model yalnız bir sonraki adımı tahmin eder; üretilen her token girdiye eklenerek uzun diziler elde edilir. Bu, sınıf sayısı sözlük boyutu kadar olan bir sınıflandırma problemidir. Bir metinden "önceki parça → sonraki token" çiftleri çok basit bir kodla çıkarılabildiği için insan etiketine gerek yoktur.
- LSTM ile karakter düzeyinde şiir ve haber üretimi, küçük veri ve küçük modelle bile lokal olarak anlamlı çıktılar verir. Bütünlük için daha uzun bağlam gerekir; Transformer bağlamı paralel alarak bunu kısmen çözer. Ölçek büyüdükçe bütünlük de sağlanır.
- Soft cosine, iki metnin farklı kelimelerle aynı şeyi söylediği durumları yakalamak için kosinüs formülünün ortasına bir kelime-kelime benzerlik matrisi M koyar. M word2vec kosinüsünden kurulursa anlamsal yakınlık, Levenshtein (düzenleme) uzaklığından kurulursa yazım farkları hesaba katılır.
- Diyalog derlemi tabanlı sistemler hafızasızdır. Diyalogda tarihçe gerektiği için RNN tabanlı kodlayıcı-çözücüler kullanılır; RNN çıkışını bir sonraki zamanda kendine girdi olarak verir, bu da bir durum (bellek) sağlar.
- Sanal asistanlar çerçeve (frame) tabanlıdır: niyet belirlenir, slotlar doldurulur, eksik slot için kullanıcıya soru sorulur. Hoca, kullanıcının söylediğinin anlaşıldığını ona hissettirmenin çok kritik olduğunu söyledi; slaytta ikinci sorudaki "İstanbul'dan Ankara'ya" kısmı kırmızıyla vurgulanmıştır (ekrandan, 01:29:20).
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.